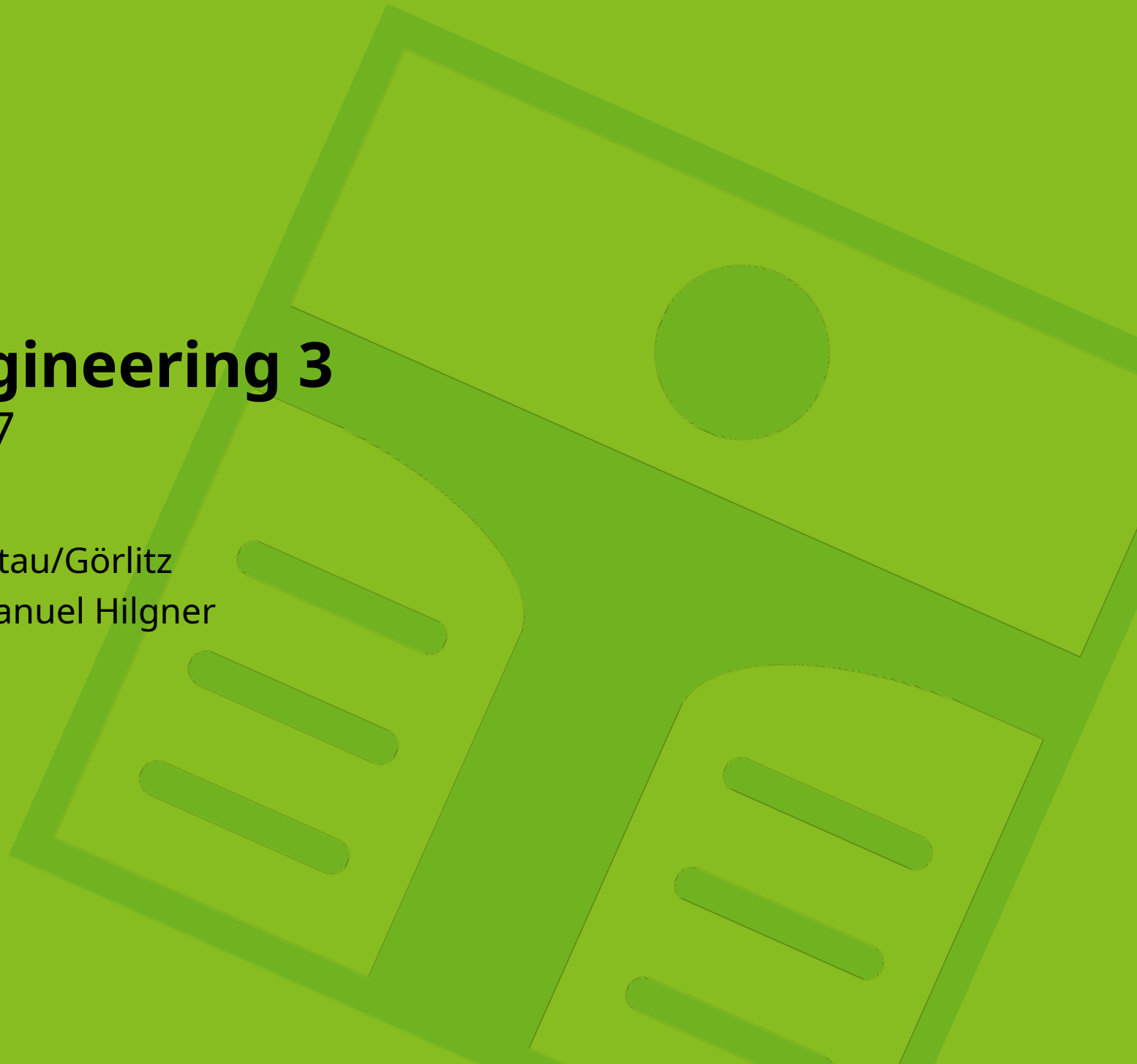


Web Engineering 3

Vorlesung 7

Hochschule Zittau/Görlitz

Christopher-Manuel Hilgner



Agenda

Vorlesung

- Error Handling in Spring
- API Dokumentation

Seminar

- Error Handling
- Build Container für Spring Anwendung
- Container für Spring Anwendung

Error Handling

Custom Template Messages

```
1 class ErrorMessageModel(  
2     var status: Int? = null,  
3     var message: String? = null  
4 )
```

 Kotlin

- status speichert den HTTP Status Code
- message speichert eine benutzerdefinierte Nachricht, die an das Frontend geschickt wird, wenn ein Error geworfen wird

Custom Template Messages

```
1  @ControllerAdvice Kotlin
2  class ExceptionControllerAdvice {
3      fun handleIllegalStateException(
4          ex: IllegalStateException
5      ): ResponseEntity<ErrorMessageModel> {
6          val errorMessage = ErrorMessageModel(
7              HttpStatus.NOT_FOUND.value(),
8              ex.message
9          )
10         return ResponseEntity(
11             errorMessage,
12             HttpStatus.BAD_REQUEST
13         )
14     }
15 }
```

Custom Template Messages

```
1 class ArticleNotFoundException(  
2     message: String  
3 ): RuntimeException(message) {  
4  
5 }
```

 Kotlin

- Es können eigene Exceptions erstellt werden
- Diese Exceptions sollten spezifische Szenarien im Backend abdecken

Custom Template Messages

```
fun handleArticleNotFoundException(ex:  
1 ArticleNotFoundException):  
  ResponseEntity<ErrorMessageModel> {  
2   val errorMessage = ErrorMessageModel(  
3     HttpStatus.NOT_FOUND.value(),  
4     ex.message  
5   )  
6   return ResponseEntity(  
7     errorMessage, HttpStatus.NOT_FOUND  
8   )  
9 }
```

[◀ Kotlin](#)

Einbindung

```
1 fun getArticle(id: String): ArticleModel {  
2     return articles.find { articleModel -> articleModel.id ==  
   id } ?: throw ArticleNotFoundException("Article not found")  
3 }
```

◀ Kotlin

Einbindung

```
1 fun createArticle(title: String): ArticleModel {  
2     val article = (articles.find { articleModel ->  
        articleModel.title == title })  
3     if (article != null) {  
4         throw IllegalStateException("Article with the same title  
            already exists")  
5     }  
6     return ArticleModel("4", title)  
7 }
```

 Kotlin

Einbindung

```
1 fun updateArticle(id: String, title: String):  
  ArticleModel {  
2     val article = (articles.find { articleModel ->  
3         articleModel.id == id  
4     } ?: throw ArticleNotFoundException("Article not found"))  
5     if (title.length > 50) {  
6         throw IllegalArgumentException("Article title too long")  
7     }  
8     article.title = title  
9     return article  
10 }
```

◀ Kotlin

ResponseStatusException Class

```
1  @PostMapping
2  fun updateArticle(
3      @RequestParam id: String,
4      @RequestParam title: String
5  ): ArticleModel {
6      try {
7          return articleService.updateArticle(id, title)
8      } catch (ex: IllegalArgumentException) {
9          throw ResponseStatusException(
10             HttpStatus.BAD_REQUEST.localizedMessage, ex
11         )
12     }
13 }
```

[◀ Kotlin](#)

ResponseStatusException Class

- Gut um Exceptions dynamisch zu handeln
- Exceptions können in der Methoden Definition des Controllers verarbeitet werden
- Es wird kein globaler Controller benötigt
- Es müssen Orte definiert werden, wo spezifische Exceptions gehandelt werden

API Dokumentation

API Dokumentation

SpringDoc

Organisation mit Tags

Controller

```
1  @Tag(  
2      name = "user_controller",  
3      description = "Controller for CRUD operations on the user"  
4  )  
5  @RestController  
6  @RequestMapping("/users")  
7  class UserController(  
8      private val userService: UserService  
9  ) {  
10     // Controller Methods  
11 }
```

[◀ Kotlin](#)

Organisation mit Tags

Controller

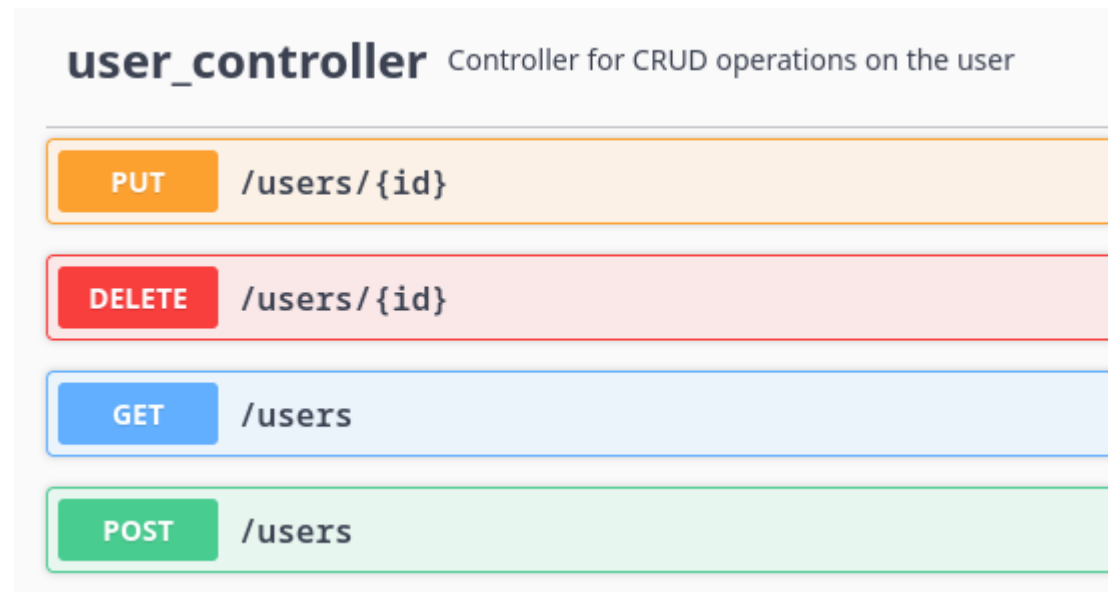


Figure 1: In der Swagger UI wird der User Controller mit den Werten aus @Tag angezeigt.

Organisation mit Tags

Methoden

```
1  @Tag(◀ Kotlin  
2      name = "getUsers",  
3      description = "Get a List of Users according to provided id and  
4                      username."  
4  )  
5  @GetMapping  
6  @ResponseStatus(HttpStatus.OK)  
7  fun getUsers(  
8      @RequestParam id: Long?,  
9      @RequestParam username: String?  
10 ): List<GetUserDto> {  
11      return userService.getUsers(id, username)  
12 }
```

Organisation mit Tags

Methoden

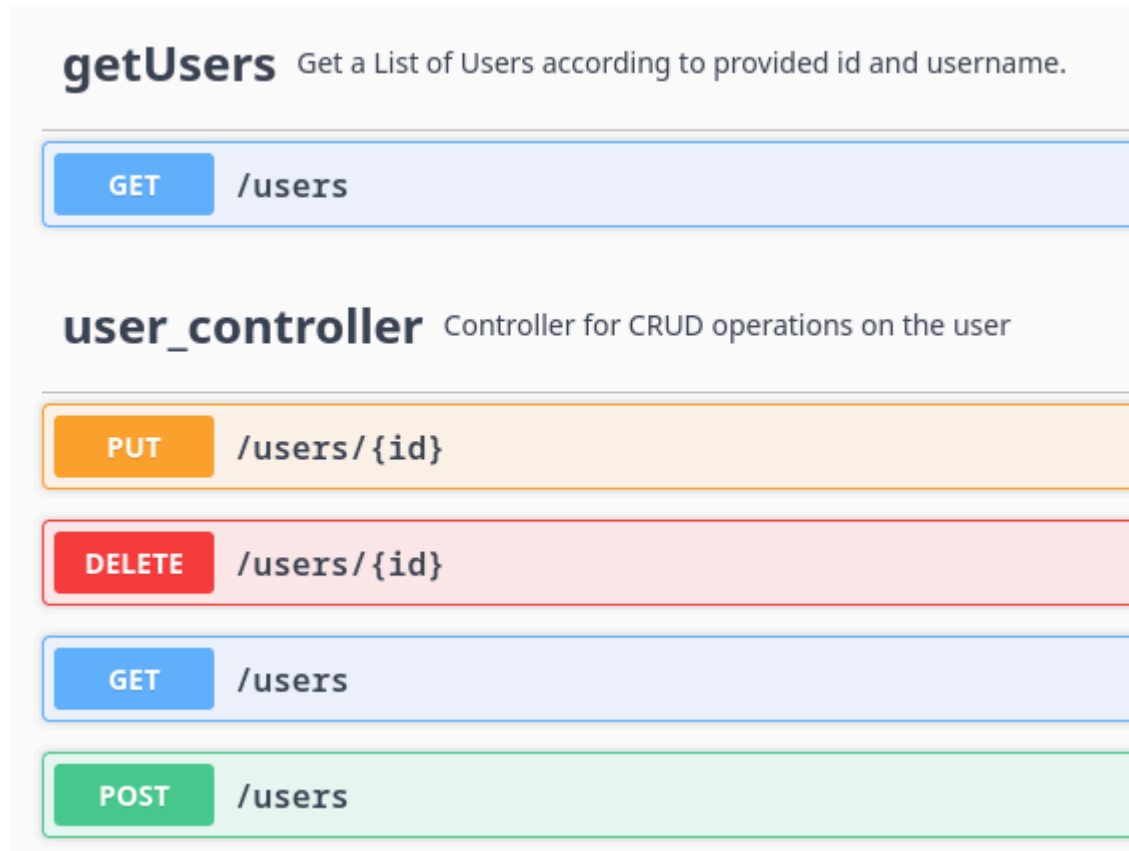


Figure 2: Methode wird mit Name und Beschreibung versehen. Sie wird dabei nicht mehr innerhalb des Controller Tags gerendert.

Organisation mit Tags

Mehrere Tags

```
1  @Tag(name = "...", description = "...")
2  @Tag(name = "GET Methods")
3  @GetMapping
4  @ResponseStatus(HttpStatus.OK)
5  fun getUsers(
6      @RequestParam id: Long?,
7      @RequestParam username: String?
8  ): List<GetUserDto> {
9      return userService.getUsers(id, username)
10 }
```

◀ Kotlin

Organisation mit Tags

Mehrere Tags

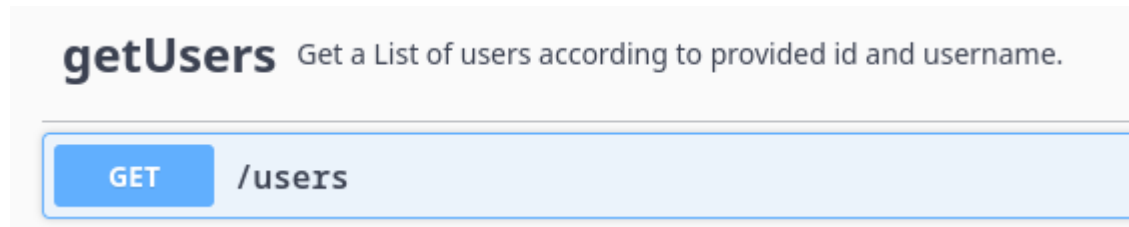


Figure 3: Eintrag in Swagger mit Name und Beschreibung

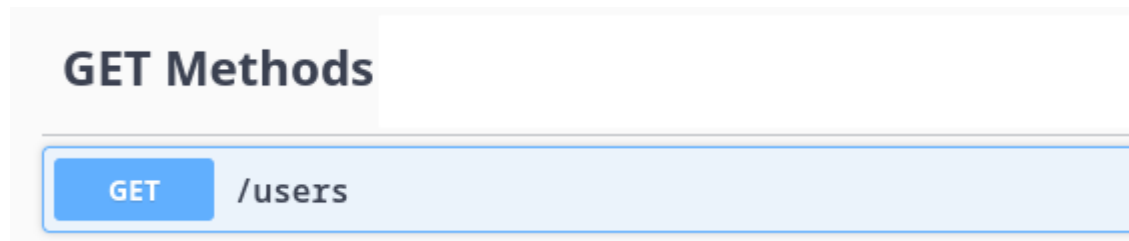


Figure 4: Eintrag in Swagger mit Name

Organisation mit Tags

Mehrere gleiche Tags an Methoden

```
1 @Tag(name = "GET Methods")
2 @GetMapping
3 @ResponseStatus(HttpStatus.OK)
4 fun getUsers(): List<GetUserDto> {
5     // Method Body
6 }
```

◀ Kotlin

```
1 @Tag(name = "GET Methods")
2 @GetMapping
3 @ResponseStatus(HttpStatus.OK)
4 fun getTodoItems(): List<GetTodoItemDto> {
5     // Method Body
6 }
```

◀ Kotlin

Organisation mit Tags

Mehrere gleiche Tags an Methoden

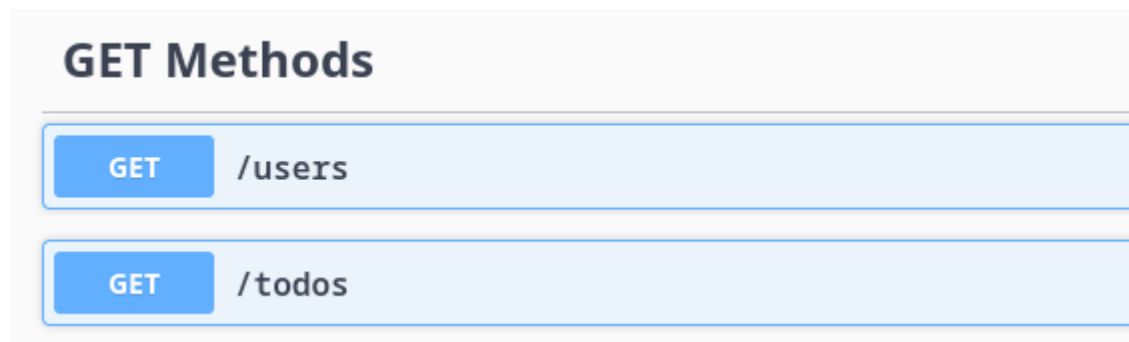


Figure 5: Methoden mit gleichen Tags werden in gemeinsame Gruppen gepackt.

Sortierung der Methoden

- Auflistung anhand der HTTP Methoden:
 1. PUT
 2. POST
 3. GET
 4. DELETE
- @Tag fügt ein Feld bei `openAPI.tags` hinzu
- Wenn Felder wie `description` oder `externalDocs` hinzugefügt werden, wird die Sortierung beeinflusst
- Wenn ein @Tag auf mehrere Operationen zutrifft, wird wieder auf die HTTP Methoden zurückgegriffen

Sortierung der Methoden

tagsSorter Property

application.properties

properties

```
1 springdoc.swagger-ui.tagsSorter=alpha
```

Alphabetische Sortierung der Tag Gruppen

application.properties

properties

```
1 springdoc.writer-with-order-by-keys=true
```

Sortierung nach Pfaden. Tags werden aber priorisiert.

Sortierung der Methoden

tags in openapi.yaml

openapi.yaml

YAML YAML

```
1 tags:
2   - name: books
3     description: Everything about your books
4     externalDocs:
5       url: http://docs.my-api.com/books.htm
6   - name: create
7     description: Add a book to the inventory
8     externalDocs:
9       url: http://docs.my-api.com/add-book.htm
```

Das books Tag wird zuerst gerendert. Danach folgt das create Tag.

Sortierung der Methoden

@OpenAPIDefinition

```
1  @OpenAPIDefinition(  
2      tags = {  
3          @Tag(  
4              name = "create",  
5              description = "Add book to inventory"  
6          ),  
7          @Tag(  
8              name = "delete",  
9              description = "Delete book from inventory"  
10         ),  
11     }  
12 )  
13 class BooksController_2 { ... }
```

[◀ Kotlin](#)

@Operation

```
1 @Operation(  
2     summary = "Get a book by its id"  
3 )  
4 public Book findById(  
5     @PathVariable long id  
6 ) {  
7     return repository.findById(id).orElseThrow(() -> new  
8         BookNotFoundException());  
9 }
```

 Java



GET

/api/book/{id} Get a book by its id

@ApiResponses

```
1  @ApiResponses( Java  
2      value = {  
3          @ApiResponse(  
4              responseCode = "200",  
5              description = "Found the book",  
6              content = {  
7                  @Content(  
8                      mediaType = "application/json",  
9                      schema = @Schema(implementation = Book.class)  
10                 )  
11             }  
12         )  
13     }  
14 )  
15 public Book findById() {}
```

@ApiResponse

GET `/api/book/{id}` Get a book by its id

Parameters

Name	Description
id ★ required integer (path)	id of book to be searched

id - id of book to be searched

Responses

Code	Description
200	Found the book

Media type
application/json

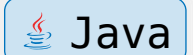
Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "title": "string",
  "author": "string"
}
```

@ApiResponse

```
1  @ApiResponse(  
2      value = {  
3          ...  
4      @ApiResponse(  
5          responseCode = "400",  
6          description = "Invalid id supplied",  
7          content = @Content  
8      ),  
9  }  
10 )  
11 public Book findById() {}
```



Java

@ApiResponse

GET /api/book/{id} Get a book by its id

Parameters

Name	Description
id ★ required integer (path)	id of book to be searched

id - id of book to be searched

Responses

Code	Description
200	Found the book

Media type
application/json

Controls Accept header.

Example Value | Schema

```
{  "id": 0,  "title": "string",  "author": "string"}
```

400	Invalid id supplied
404	Book not found

MENSA IST VORBEI